

Busca binária

Realizar uma busca de um valor alvo em um vetor pode ser feito através de uma busca linear, onde percorremos cada elemento até encontrar o valor alvo. A complexidade é de $O(n)$, e realmente não há como melhorar isso para um vetor arbitrário.

Mas e se o vetor estiver ordenado?

Nesse caso, uma consulta ao elemento que está no meio desse vetor nos dá uma informação valiosa sobre qual metade do vetor nosso alvo pode estar.

A busca binária consiste em:

1. Olhar para o elemento do meio do vetor. Se esse é o alvo, então encontramos o elemento.
2. Se o alvo é menor que o elemento do meio, então o alvo só pode estar na esquerda.
3. Se o alvo é maior que o elemento do meio, então o alvo só pode estar na direita.
4. Repetir esse processo para a metade do vetor que contém o alvo.

A cada iteração da busca binária, o tamanho do vetor que precisamos procurar é reduzido pela metade. Dessa forma, temos uma complexidade $O(\log n)$.

Implementação:

Nesse código, retornamos o índice do elemento alvo, ou -1 caso esse alvo não esteja no vetor.

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100010;
int V[MAXN], n, q;

int busca_binaria(int alvo){
    int esq = 0;
    int dir = n - 1;

    while(esq <= dir){
        int meio = (esq + dir) / 2;
        if(V[meio] == alvo){
            return meio;
        } else if(alvo < V[meio]){
            dir = meio - 1;
        } else {
            esq = meio + 1;
        }
    }
    return -1;
}

int main(){
    cin >> n >> q;
    for(int i = 0; i < n; i++){
```

```
        cin >> V[i];
    }
    sort(V, V + n);

    for(int i = 0; i < q; i++){
        int alvo;
        cin >> alvo;
        cout << busca_binaria(alvo) << endl;
    }
}
```

Nesse código acima, a busca binária é utilizada para resolver Q consultas de busca do índice, alcançando uma complexidade de $O(q \cdot \log(n))$.

Busca binária na resposta

A busca binária não está limitada a apenas encontrar elementos em vetores ordenados. A busca pode ser realizada em qualquer espaço desde que este esteja ordenado.

Formalmente, se uma função qualquer $F(x)$ é monótona, ou seja, é sempre crescente ou sempre decrescente, podemos aplicar a busca binária para encontrar valores de x que satisfaçam certas condições.

Considere o problema de calcular a raiz quadrada inteira de um número N . A raiz quadrada inteira é o maior inteiro x tal que $x^2 \leq N$.

Por exemplo, veja os passos para calcular a raiz quadrada inteira de 100:

- O valor 50^2 é a raiz de 100? Não, $50^2 = 2500 > 100$
- O valor 25^2 é a raiz de 100? Não, $25^2 = 625 > 100$
- O valor 12^2 é a raiz de 100? Não, $12^2 = 144 > 100$
- O valor 6^2 é a raiz de 100? Não, $6^2 = 36 < 100$
- O valor 9^2 é a raiz de 100? Não, $9^2 = 81 < 100$
- O valor 10^2 é a raiz de 100? Sim, $10^2 = 100 = 100$

Essa ideia pode ser implementada da seguinte forma em $O(\log N)$:

```
typedef long long ll;

int raiz_inteira(int n){
    ll esq = 0;
    ll dir = n;
    ll resposta = 0;

    while(esq <= dir){
        ll meio = (esq + dir) / 2;
        if(meio * meio <= n){
            resposta = meio;
            esq = meio + 1;
        } else {
            dir = meio - 1;
        }
    }
}
```

```
    }  
    return resposta;  
}
```

Nesse exemplo, fizemos uma busca para encontrar o maior valor de x que satisfaça a função $F(x) = 1$, onde $F(x) = 1$ se $x^2 \leq N$ e $F(x) = 0$ caso contrário. A função é monótona.

Desde que a solução do problema seja crescente ou decrescente, podemos aplicar a busca binária para encontrar a resposta.